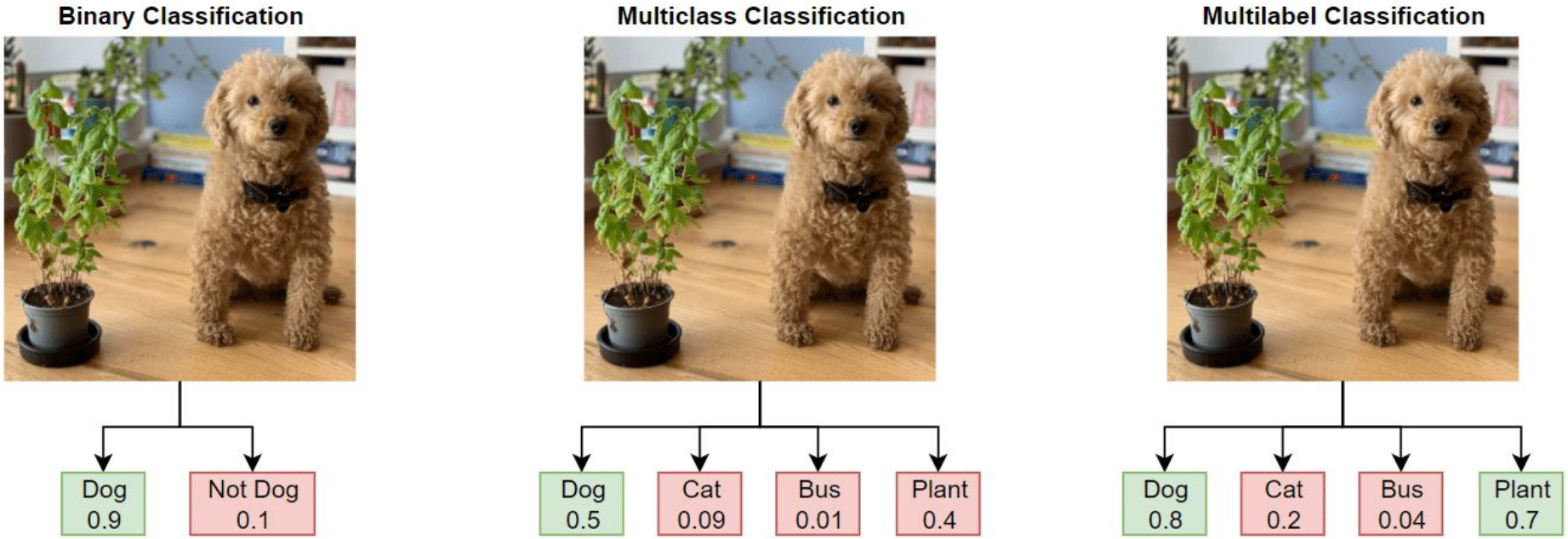
# Regression

Lecturer: Albert Yang (楊景明)

# Quick review

- Split dataset
- Random sampling, stratified sampling
- K-fold cross validation
- Binary, multiclass, multilabel, multioutput classification
- Confusion matrix, precision, recall, f1, ROC, AUC

# Binary, multiclass, and multilabel




Source: https://la.mathworks.com/help/deeplearning/ug/multilabel-image-classification-using-deep-learning.html

# The details behind model training

- So far, we have treated our ML model as a black box
  - Ex: Image -> model -> digit

# The details behind model training

- So far, we have treated our ML model as a black box
  - Ex: Image -> model -> digit
- Why should we understand the details behind?

# The details behind model training

- So far, we have treated our ML model as a black box
  - Ex: Image -> model -> digit
- Why should we understand the details behind?
  - Select appropriate models
  - Train your models in a proper way
  - Tune hyperparameters
  - Error analysis
  - Prerequisite for the neural networks!

# Regression tasks

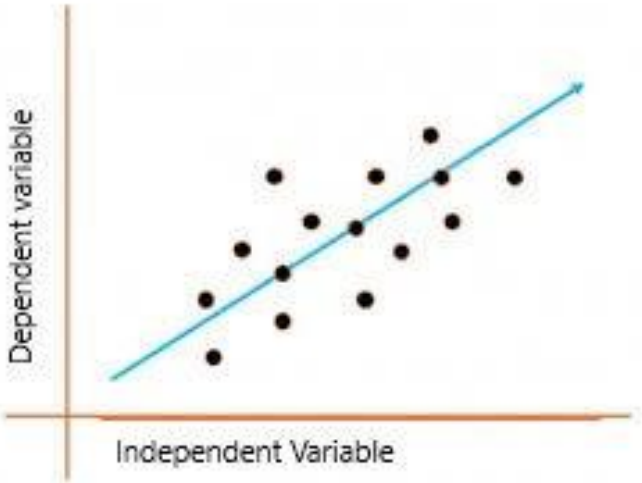
- Predict continuous values
  - Housing values, midterm exam scores, etc.
- Linear regression
  - Linear relationship
- Polynomial regression
  - Non-linear relationship

# Linear regression

Prediction = weighted sum of the features + bias

$$y_{pred} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$$

- $y_{pred}$ : predicted value
- $n$: number of features
- $x_i$: value of $i$ th feature
- $\theta_j$: $j$ th model parameter (weight)
- $\theta_0$: bias




Source: https://www.analyticsvidhya.com/blog/2021/10/everything-you-need-to-know-about-linear-regression/

# Evaluating regression models

- Commonly used error metrics
  - Mean Squared Error (MSE)
  - Root Mean Squared Error (RMSE)
  - Mean Absolute Error (MAE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

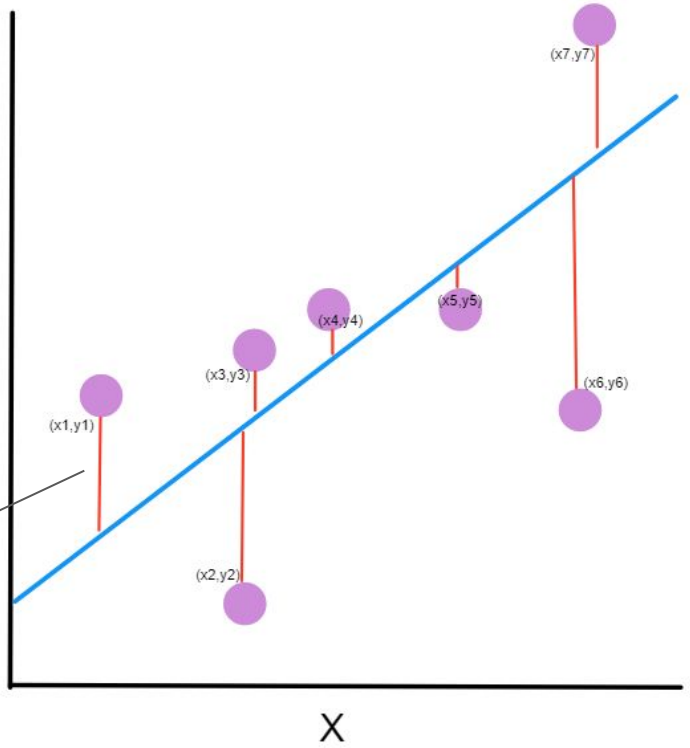
risidual

$\text{MSE}$ = mean squared error

$n$ = number of data points

$Y_i$ = observed values

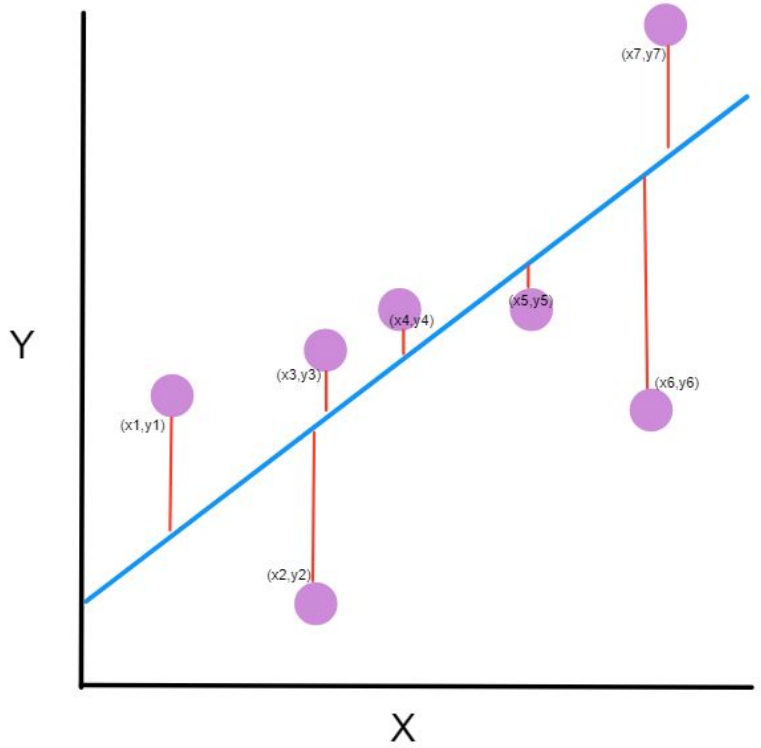
$\hat{Y}_i$ = predicted values



# Evaluating regression models

- Commonly used error metrics
  - Mean Squared Error (MSE)
  - Root Mean Squared Error (RMSE)
  - Mean Absolute Error (MAE)

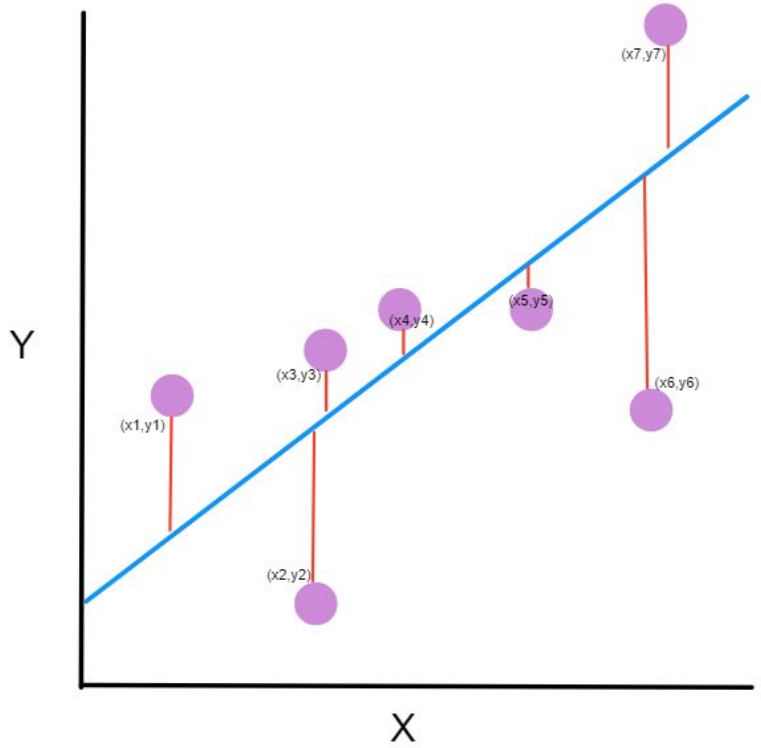
$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$



# Evaluating regression models

- Commonly used error metrics
  - Mean Squared Error (MSE)
  - Root Mean Squared Error (RMSE)
  - Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} | y_i - \hat{y}_i |$$



# Gradient descent

What happens during training?

- Randomly intitalize the model parameters (weights)
- Pass a sample (or several) to the model and get predicted value
- Calculate the errors
- Update the parameters using gradient descent
- Pass more samples

# Imagine you are lost in a foggy mountain

- You can't see anything. All you can do is feel the slope of each step

# Imagine you are lost in a foggy mountain

- You can't see anything. All you can do is feel the slope of each step
- You want to go down the mountain as soon as possible

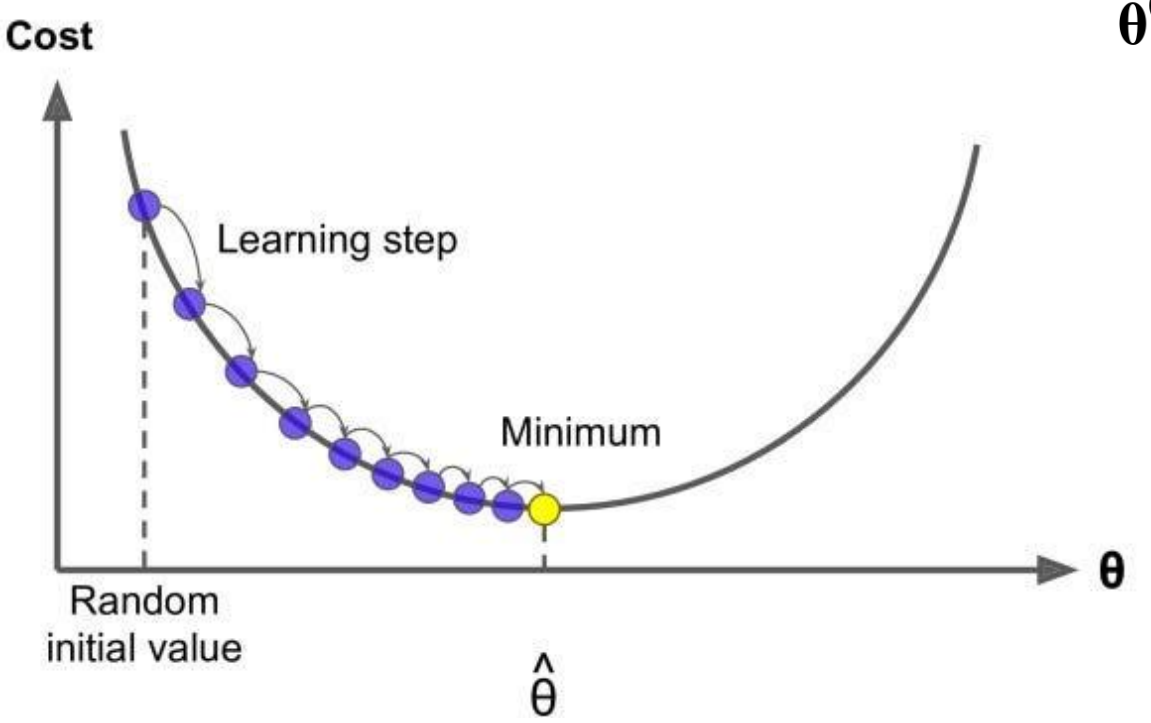
# Imagine you are lost in a foggy mountain

- You can't see anything. All you can do is feel the slope of each step
- You want to go down the mountain as soon as possible
- One ideal way is to go with the steepest slope

# Imagine you are lost in a foggy mountain

- You can't see anything. All you can do is feel the slope of each step
- You want to go down the mountain as soon as possible
- One ideal way is to go with the steepest slope
- That is how gradient descent works!

# How parameters are updated

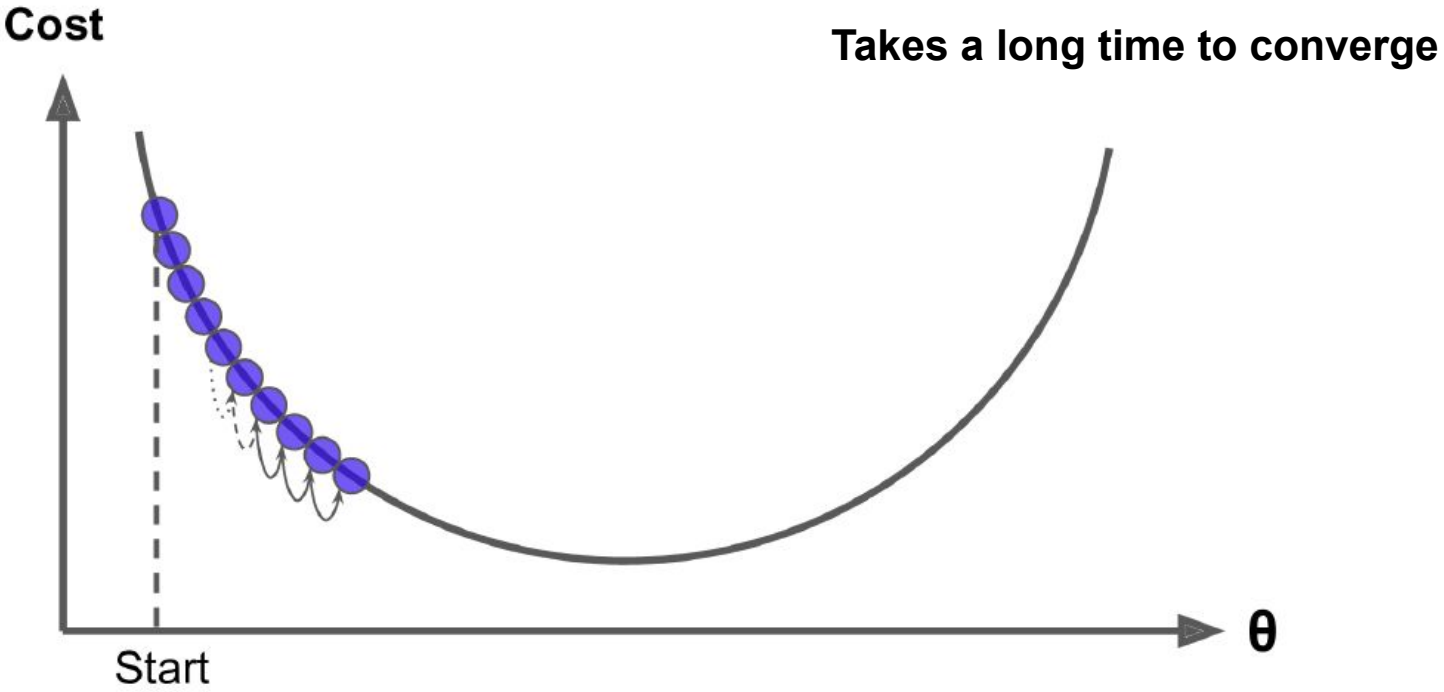



$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \, \text{MSE}(\boldsymbol{\theta})$$

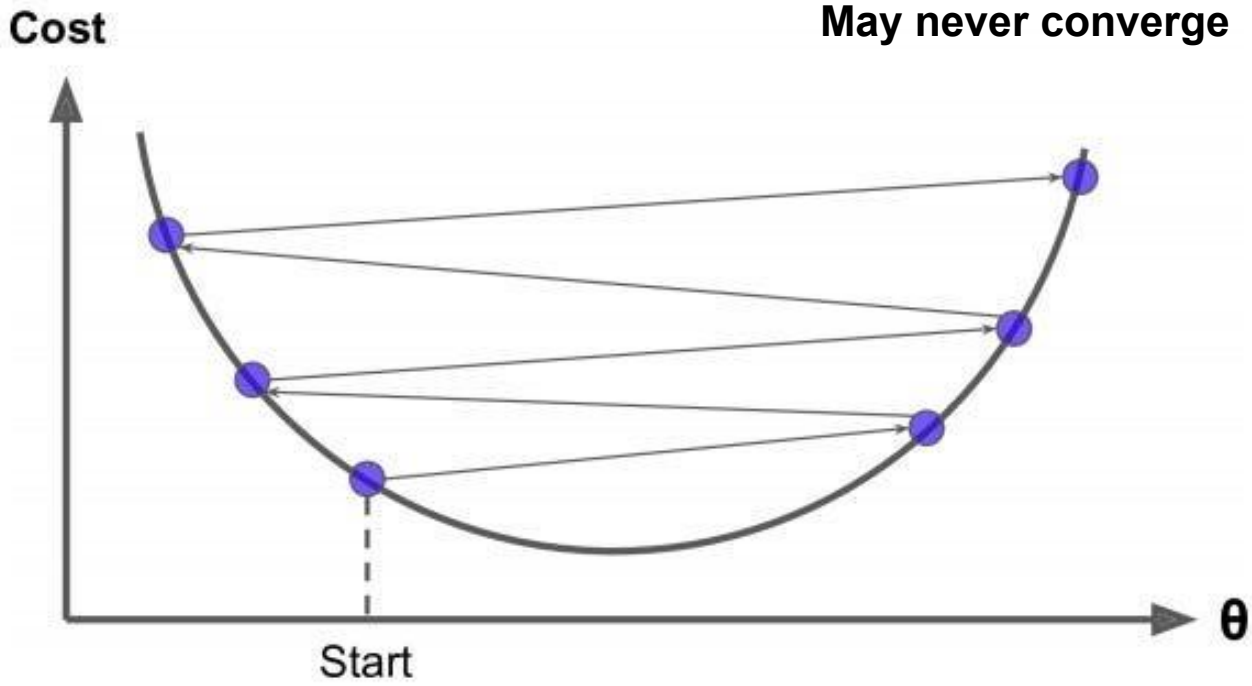
learning rate gradient

Learning step

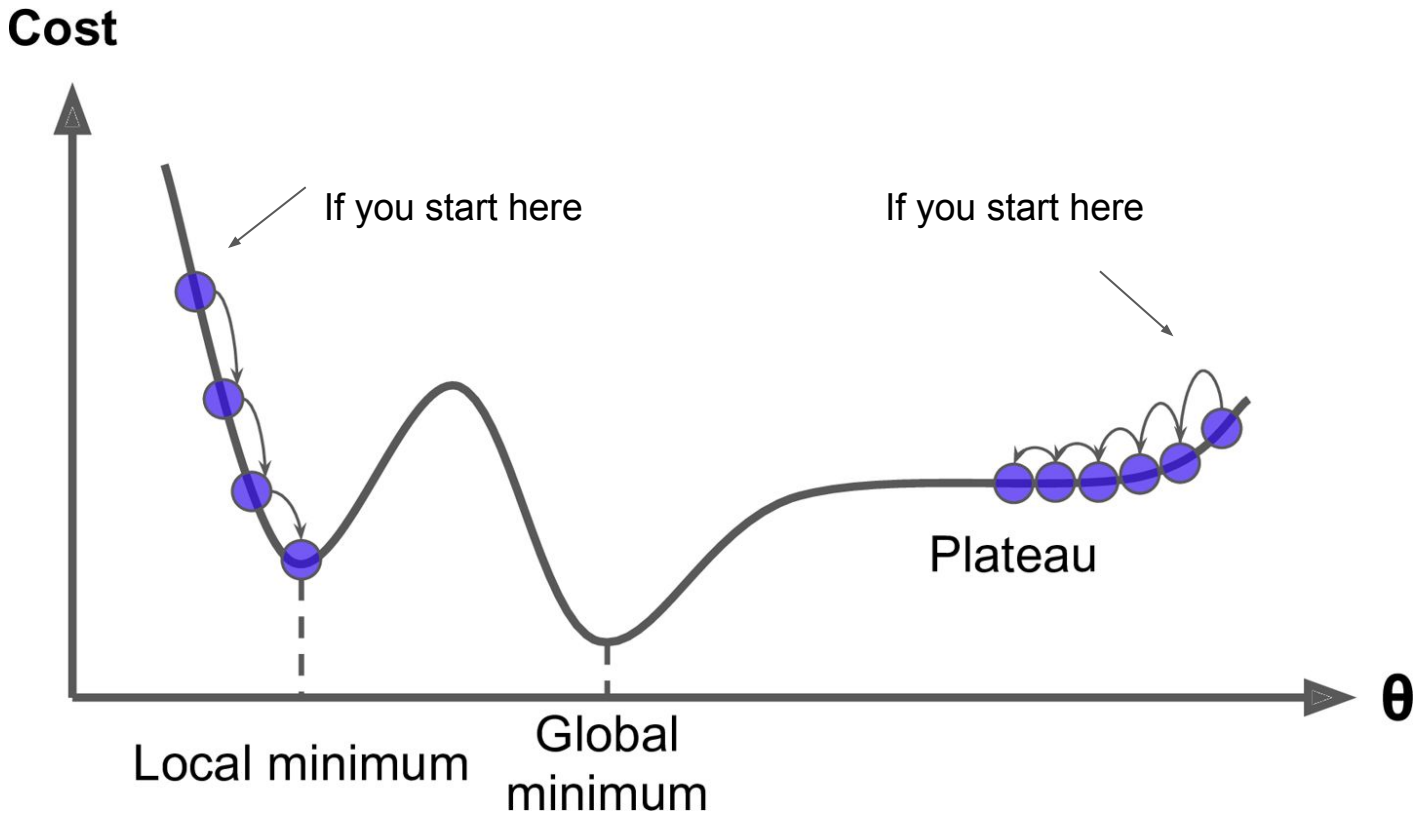
# If the learning rate is too small



# If the learning rate is too large



# Your loss function might not look like a bowl



# Calculating gradient

- Gradient represents how much the cost function will change if you change $\theta_j$ just a little bit (a.k.a partial derivative)
- Need to calculate the gradients of each parameter
- *What is the slope of the mountain under my feet if I face east? (what about north, west?)*
- Types of gradient descent
  - Batch gradient descent
  - Stochastic gradient descent
  - Mini-batch gradient descent

# Batch gradient descent

dummy feature, set to 1

$\theta = [\theta_0, \theta_1, \theta_2]$ $\quad \theta^T = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$ $\quad x = [x_0, x_1, x_2]$

$\theta^T x^{(i)}$
$= \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$
$= \theta_0 + \theta_1 x_1 + \theta_2 x_2$

The partial derivative of the cost function with regards to parameter θj =

$$\frac{\partial}{\partial \theta_j} \mathrm{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) \, x_j^{(i)}$$

residual of ith sample

j th feature of i th sample

Calculating gradient step-by-step: https://www.youtube.com/watch?v=sDv4f4s2SB8

# Step-by-step (skip)

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

- Step 1: Start with the definition of MSE
- Step 2: Take the partial derivative with respect to θ□

  Let $u^{(i)} = \theta^T x^{(i)} - y^{(i)}$, so MSE becomes

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(u^{(i)}\right)^2$$

  Applying the chain rule with respect to θ□:

$$\frac{\partial}{\partial\theta_j}\text{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} 2u^{(i)} \cdot \frac{\partial u^{(i)}}{\partial\theta_j}$$

# Step-by-step (skip)

- Step 3: Compute the partial derivative
  Since $u^{(i)} = \theta^T x^{(i)} - y^{(i)} = \theta_0 x_0^{(i)} + \theta_1 x_1^{(i)} + \cdots + \theta_j x_j^{(i)} + \cdots - y^{(i)}$
  $y^{(i)}$ is a constant, so it vanishes under differentiation. In the expanded sum, only the term $\theta_j x_j^{(i)}$ depends on θ□; every other term has zero derivative with respect to θ□. So

$$\frac{\partial u^{(i)}}{\partial \theta_j} = x_j^{(i)}$$

- Step 4: Substitute back

$$\frac{\partial}{\partial \theta_j} \mathrm{MSE}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} 2 \left( \boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)} = \frac{2}{m} \sum_{i=1}^{m} \left( \boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

# Batch gradient descent

$$X = \begin{bmatrix} x^1_0 & x^1_1 & x^1_2 \\ x^2_0 & x^2_1 & x^2_2 \\ \cdots & & \end{bmatrix} \qquad X\theta = \begin{bmatrix} y^1 \\ y^2 \\ y^3 \end{bmatrix}$$

$$\theta = [\theta_0, \theta_1, \theta_2]$$

You can compute them all in one go

$$X^T = \begin{bmatrix} x^1_0 & x^2_0 & \\ x^1_1 & x^2_1 & \cdots \\ x^1_2 & x^2_2 & \end{bmatrix}$$

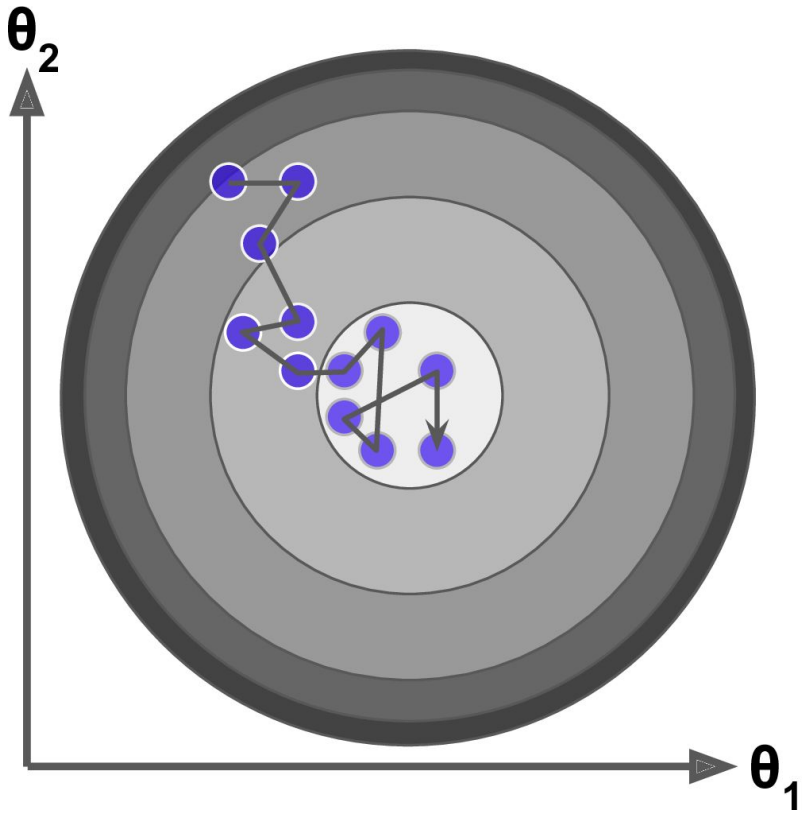
$$\nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \operatorname{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \operatorname{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \operatorname{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

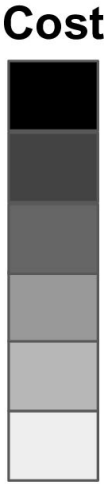
To obtain the scalar value

# Stochastic gradient descent (SGD)

- Batch gradient descent uses the whole training set to compute the gradients at every step
- Very slow when the training set is large
- Stochastic gradient descent just picks a random instance in the training set at every step and computes the gradients based only on that single instance
- Makes the algorithm much faster and makes it possible to train on huge training sets

# Stochastic gradient descent




However

- The cost function will bounce up and down, decreasing only on average.
- It will end up very close to the minimum, but once it gets there it will continue to bounce around
- The final parameter values are good, but not optimal.

# Stochastic gradient descent

- SGD has a better chance of finding the global minimum than Batch Gradient Descent does
- However, can never settle at the minimum
- One solution to this dilemma is to gradually reduce the learning rate
- The function that determines the learning rate at each iteration is called the learning schedule

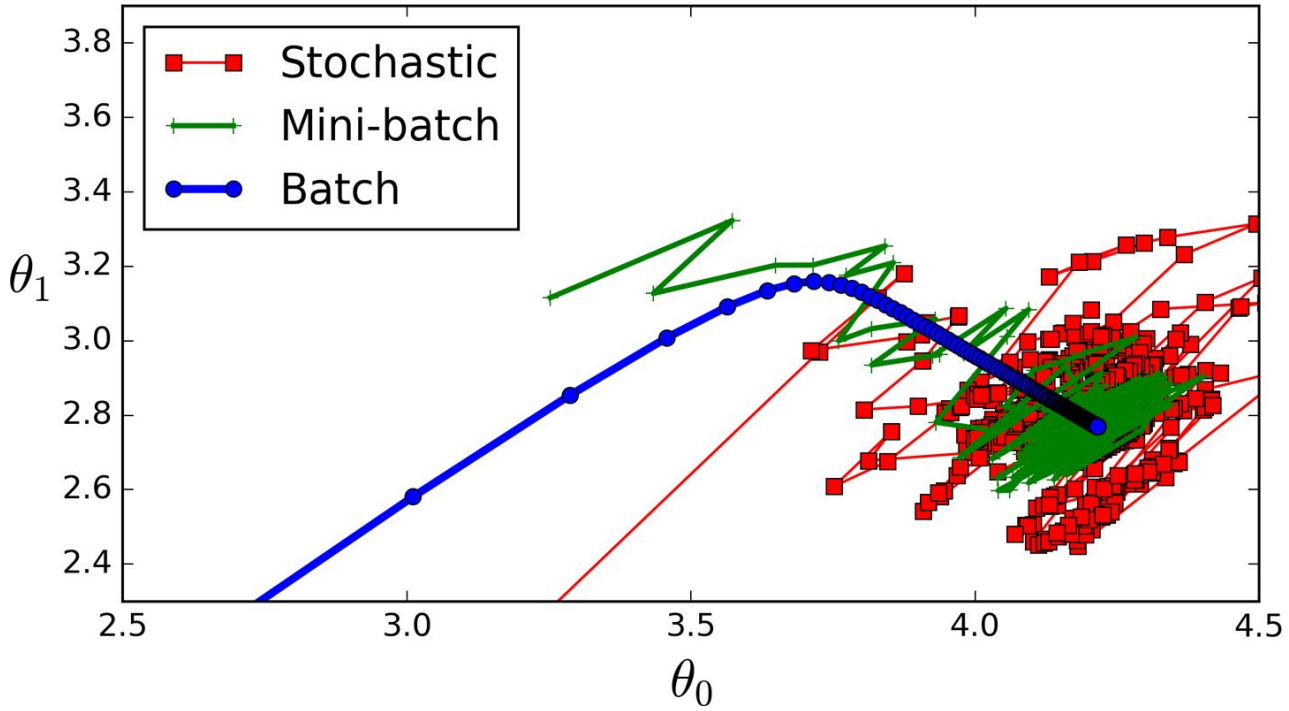
Note:

- If we have 100 samples, SGD will randomly select one sample at each step → 100 steps
- Once the SGD select 100 samples → 1 epoch

# Mini-batch gradient descent

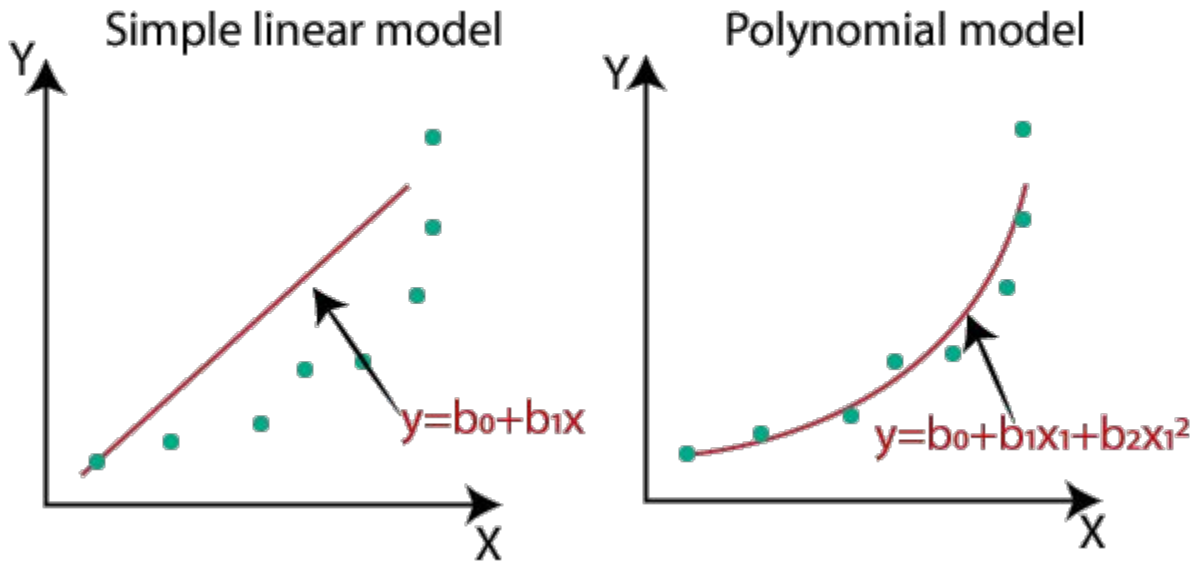
- In between the batch GD and stochastic GD
- At each step, mini-batch GD computes the gradients on small random sets of instances
- Compared to stochastic GD, mini-batch GD can get a performance boost from hardware optimization of matrix operations, especially when using GPUs

- Keep in mind that batch GD takes a lot of time to take each step
- Stochastic GD and Mini-batch GD would also reach the minimum if you used a good learning schedule

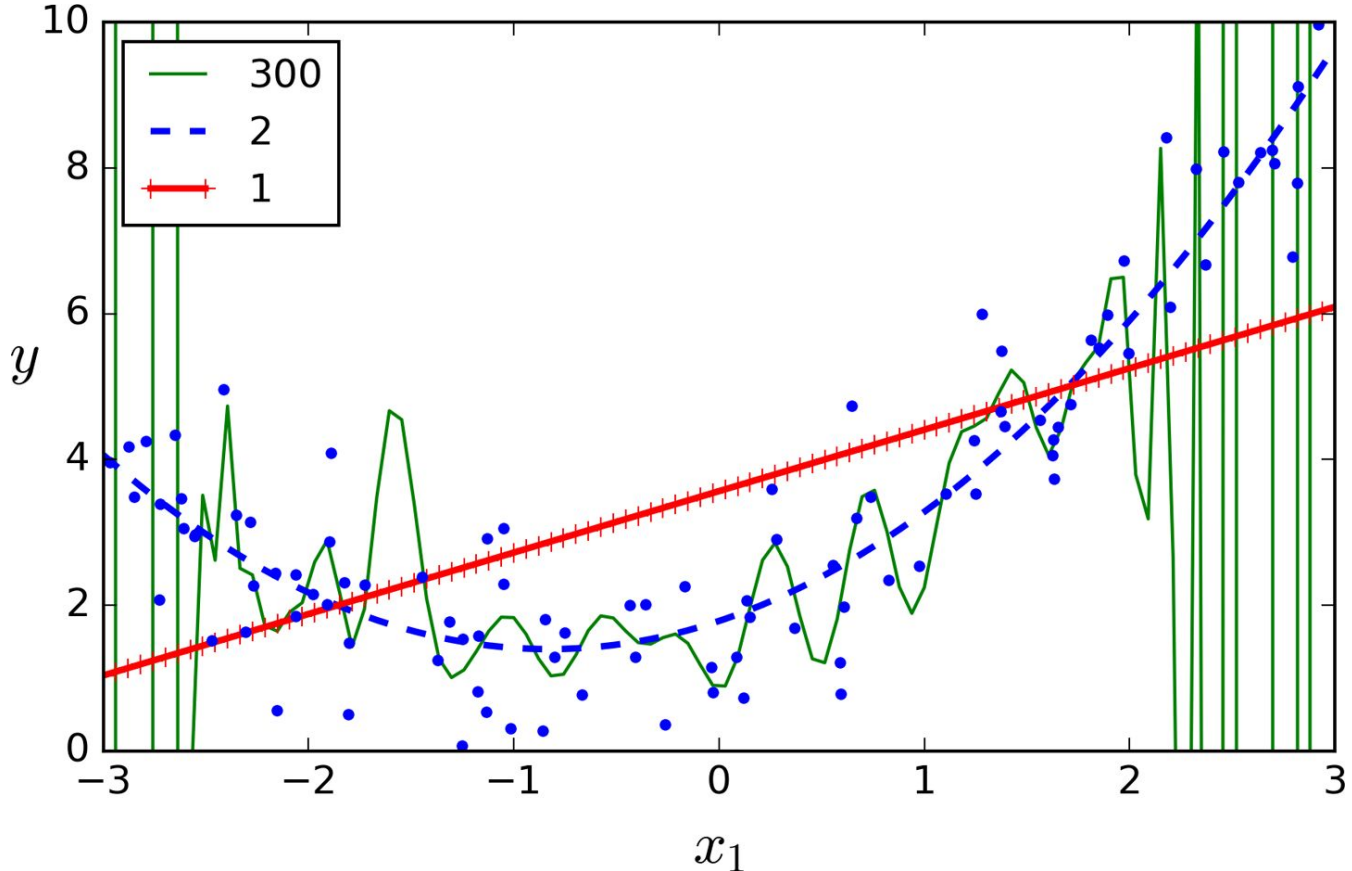


# Polynomial regression

What if you have non-linear data?



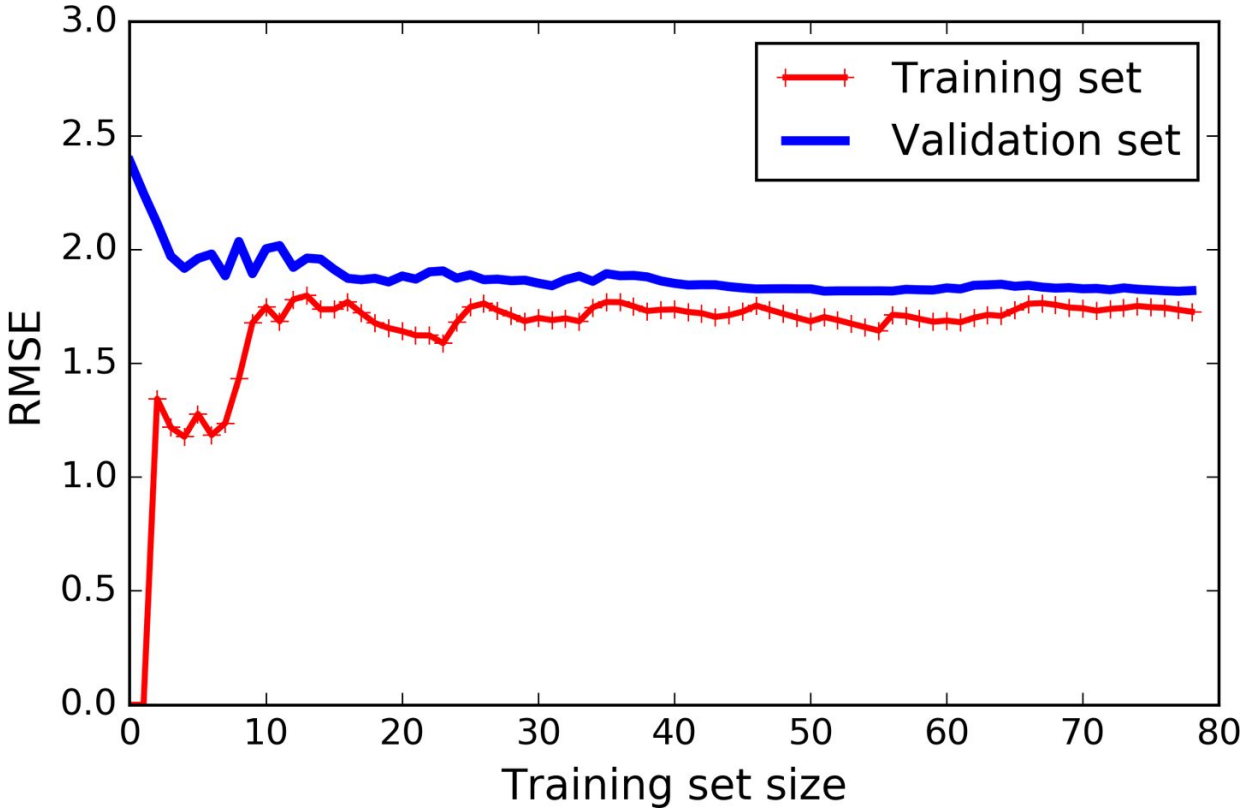
# Be aware when choosing the *degree* parameter



# How to determine the model complexity

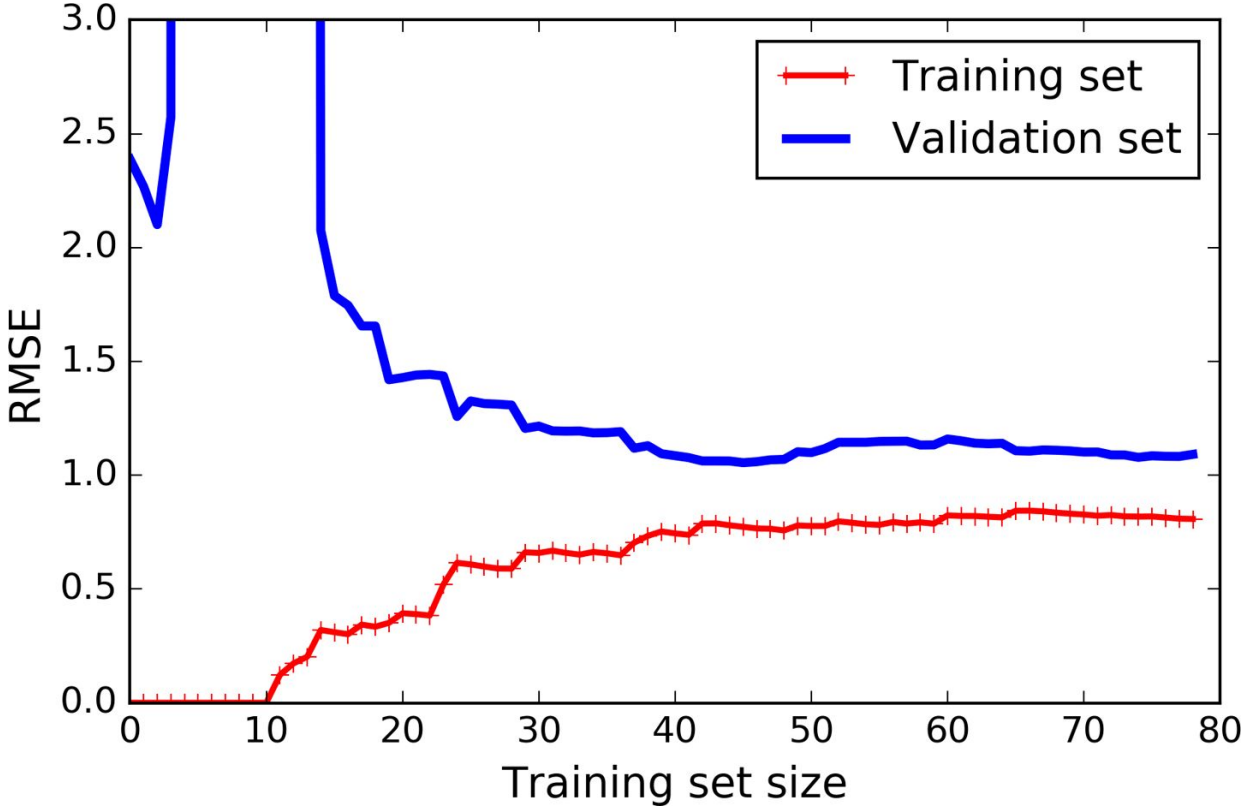
- We can use cross validation
  - If the training result is good, but testing (validation) result is poor → overfitting
  - If both training and testing (validation) result are poor → underfitting
  - Cross validation helps in assessing how well the model generalizes
- Another approach is to plot a learning curve

# Learning curve




- Underfit
- Instead of more training data, you should consider a more complex model

# Learning curve



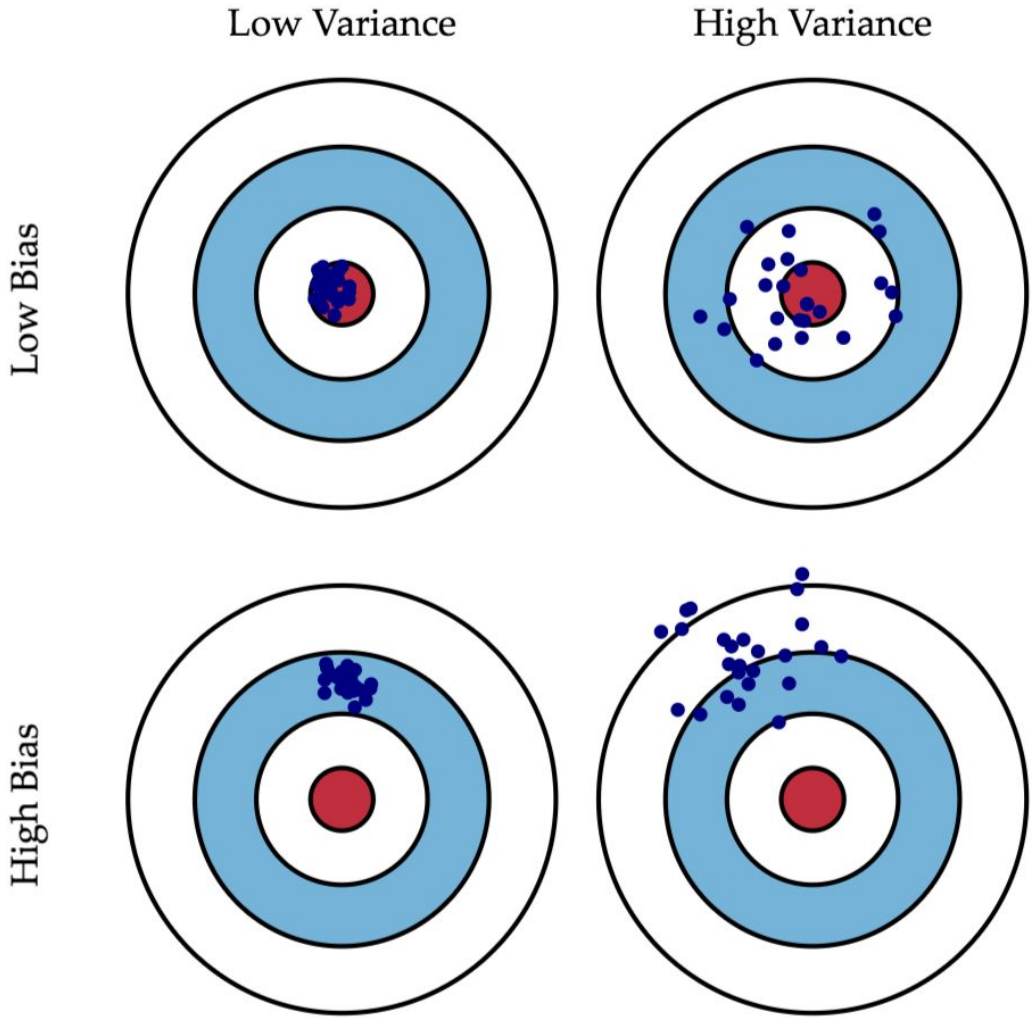

- Overfit
- You can provide more training data

# The Bias/Variance Tradeoff

- Total error = Bias + Variance
- Bias
  - Due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data
- Variance
  - Model's sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance, and thus to overfit the training data

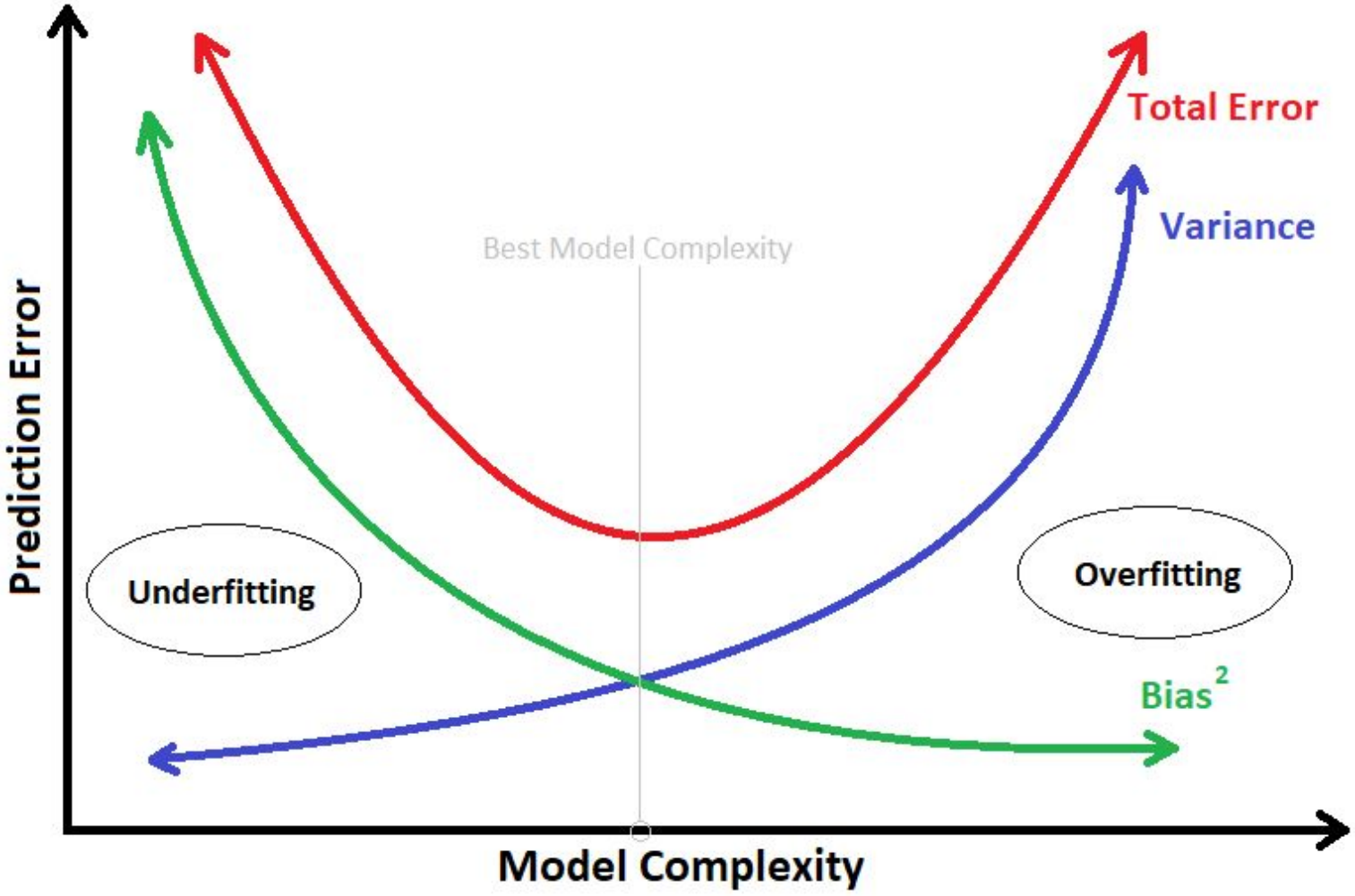
Idea model

Complex model




Wrong model

What are you doing?


Total Error
Variance
Best Model Complexity
Prediction Error
Underfitting
Overfitting
Bias$^2$
Model Complexity

# Regularization

- A good way to reduce overfitting is to regularize the model (i.e., to constrain it)
  - The fewer degrees of freedom it has, the harder it will be for it to overfit the data
  - Ex: a simple way to regularize a polynomial model is to reduce the number of polynomial degrees.
- Regularization regression
  - Ridge
  - Lasso
  - Elastic net

# Ridge regression

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^{n} \theta_i^2$$

Regularized total loss | Total loss | L2 panelty

- If $\alpha$ = 0: normal linear regression
- If $\alpha$ is large: every weights will be very close to zero → a horizontal straight line
- Make sure to scale the features before regularization
  ex: MinMax scaler, Standard scaler

# L2 panelty

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$



Trying to minumize every weights

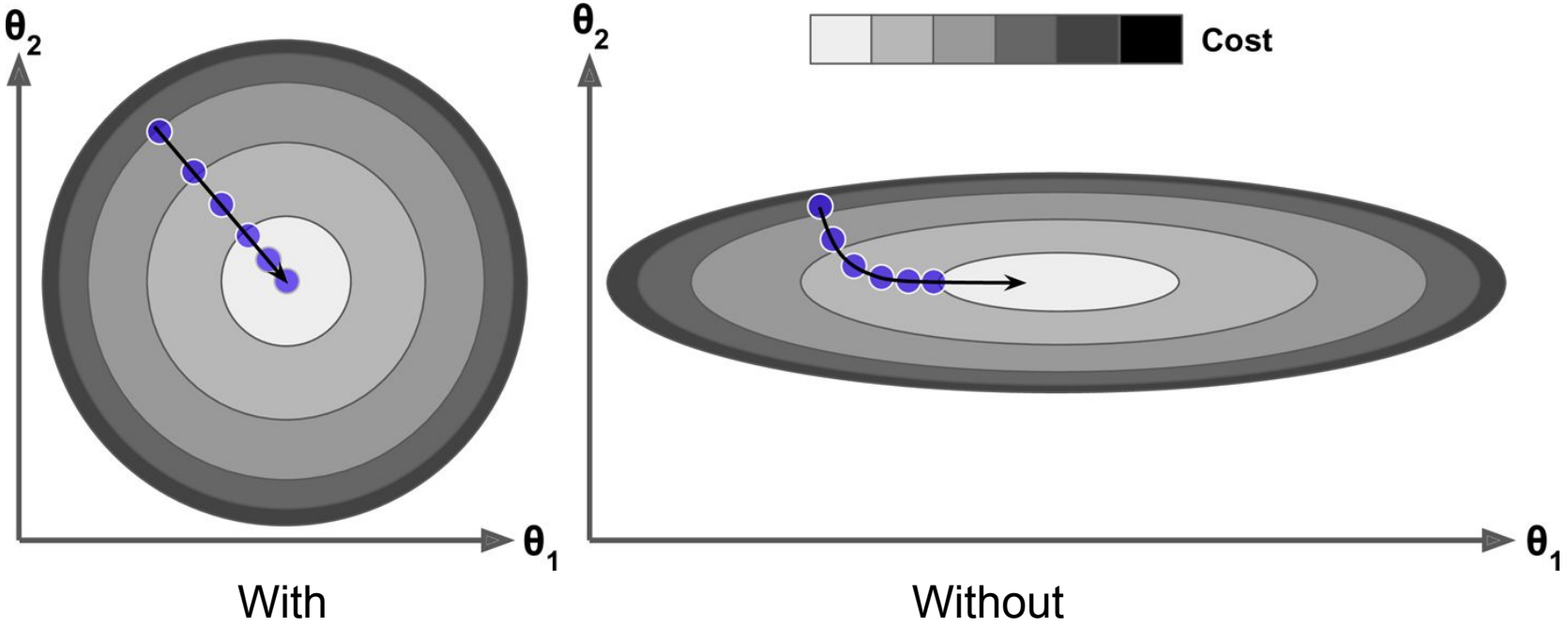
# Feature scaling

- Large differences in scales can lead to bad performance
  - Total studying minutes: 0 to 10,000 mins
  - Midter exam scores: 0 to 100
  - Feature with larger scale tend to have a greater impact on prediction, especially those that rely on distance measures like KNN

# Feature scaling

- Consistency in feature importance
  - Feature scaling can help models that depend on the relative importance of features, such as regularization methods (L1, L2 regularization).
- Feature scaling allows the model to interpret these features on the same scale
- Faster convergence in optimization algorithms

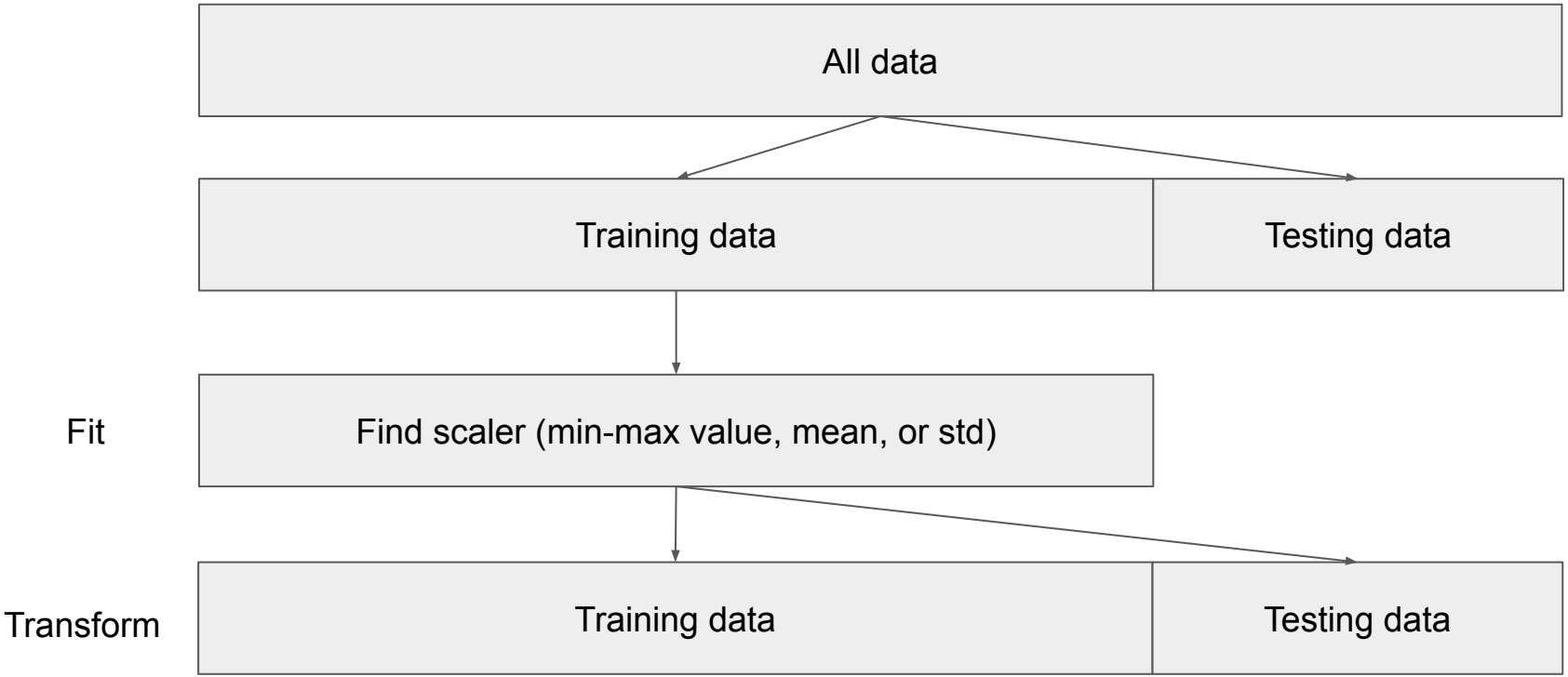
# Convergence Speed with/without Feature Scaling



# Feature scaling approaches

- Min-max scaling: (x - min) / (max - min)
- Standardization: (x - mean) / standard deviation

# The proper way of performing feature scaling



# Lasso regression

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^{n} |\theta_i|$$

Regularized total loss | Total loss | L1 panelty

- If $\alpha$ = 0: normal linear regression
- If $\alpha$ is large: evey weights will be set to zero → a horizontal straight line
- Make sure to scale the features before regularization
  ex: MinMax scaler, Standard scaler

# L1 panelty

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

Trying to eliminate the weights of the least important features (i.e., set them to zero)

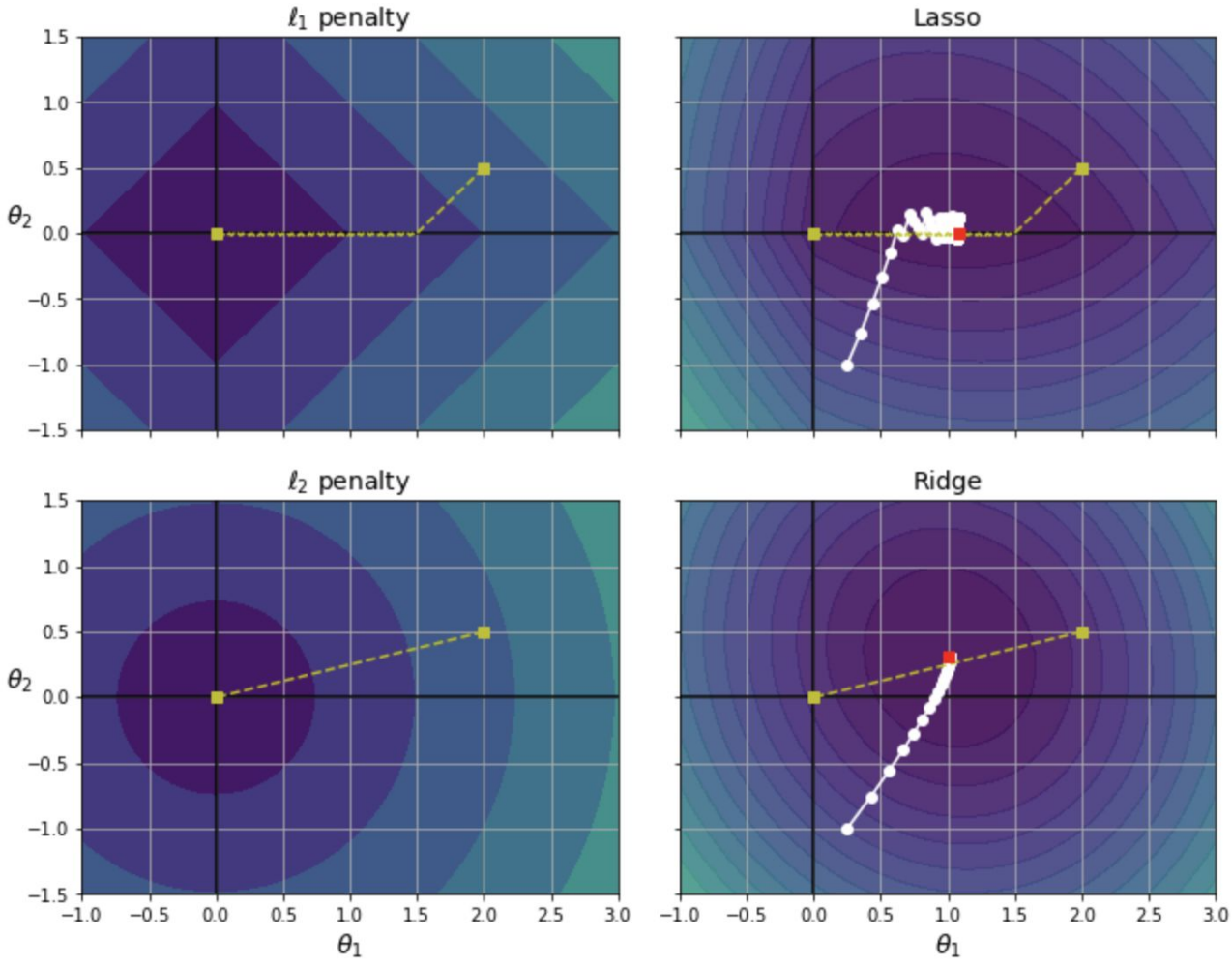
# L1 panelty

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

Trying to eliminate the weights of the least important features (i.e., set them to zero)



Feature selection


$\ell_1$ penalty
Lasso
$\ell_2$ penalty
Ridge
$\theta_1$
$\theta_2$

# Elastic net

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{i=1}^{n} |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^{n} \theta_i^2$$

- The middle ground between Ridge Regression and Lasso Regression
- If *r* is 0 → Rigde  If *r* is 1 → Lasso
- Make sure to scale the features before regularization
  ex: MinMax scaler, Standard scaler

# Which regularization should I use?

- In general, regularization will lead to better results
- If you have no idea about your data
  - Ridge regression is a good start
- If you know that only a few features are important
  - Lasso regression
- In general elastic net is better than lasso (unless each feature is independent of other features)

# Early stopping

A simple regularization by stopping training as soon as the validation error reaches the minimum

For Stochastic GD and mini-batch GD, stop only after the validation error has been above the minimum for some time, then roll back the model parameters

